Lab 7: Fuzz Testing

Traffic Fine Calculator (L6_L7_27) — Property-based fuzz testing with Hypothesis

Field	Details
Student name	Leesha R
SRN / USN	_____ (to be filled in)
Section / Semester	_____ (to be filled in)
Course	_____ (to be filled in)
Faculty	_____ (to be filled in)
Repository	github.com/SETAPESU26/L6_L7_27_traffic_fine (folder: lab7)
Date	11 November 2026

1. Objective of fuzz testing

Fuzz testing feeds a program large numbers of automatically generated, often unexpected inputs (empty strings, None, wrong types, Unicode, huge or negative numbers, malformed formats) and checks that it never crashes and always produces correct results. The objective of this lab was to fuzz the three functions in `traffic_fine.py` with Hypothesis, expose the crashes and wrong results in the original code, fix them with proper input validation, and show that the fixed code passes the full fuzz test suite.

2. Tools and technologies

Tool	Version	Purpose
Python	3.13.18	Language of the application and tests
Hypothesis	6.170.15	Property-based / fuzz test generation and shrinking (@given, strategies)
pytest	9.2.0	Test runner (python -m pytest -v)
Git	-	Version control; original code preserved in commit 09d47d2

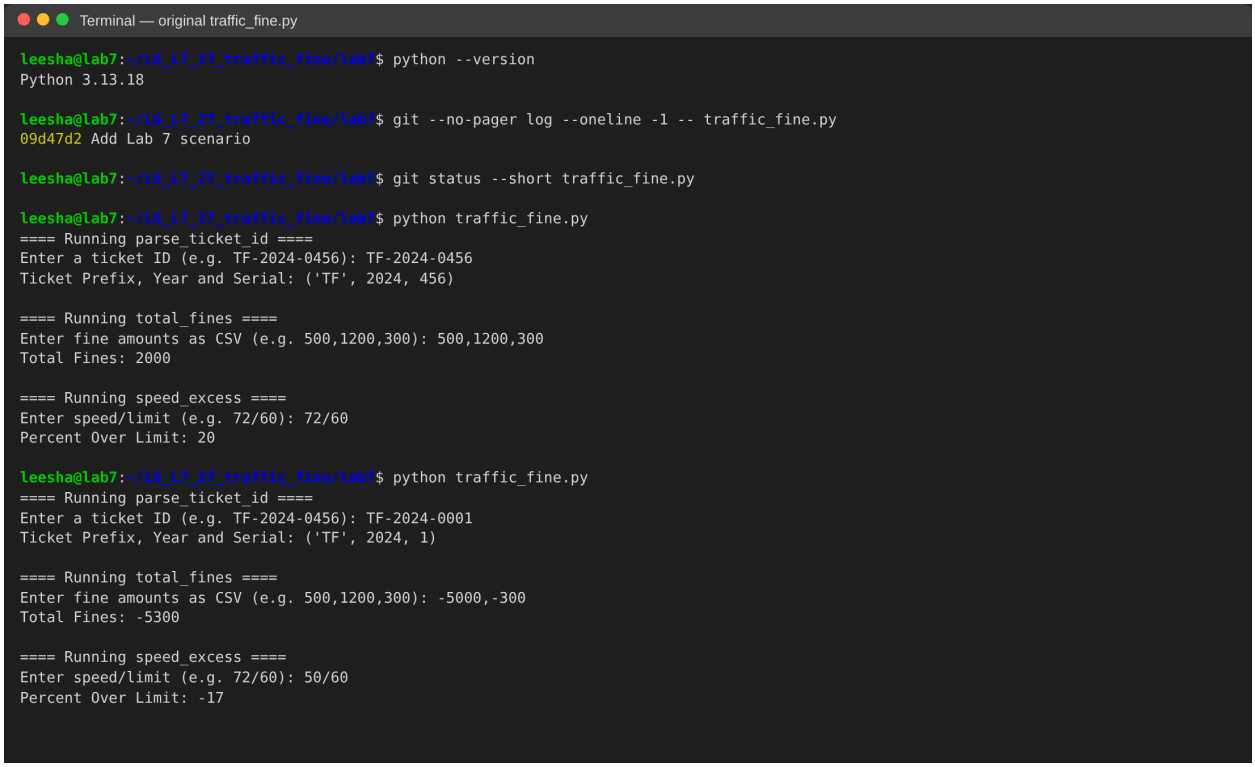
3. The three functions under test

Function	Input (example)	Intended behaviour
parse_ticket_id	'TF-2024-0456'	Split into PREFIX-YEAR-SERIAL and return ('TF', 2024, 456). Prefix = letters, year = 4 digits, serial = digits.
total_fines	'500,1200,300'	Add comma-separated non-negative fine amounts; total capped at 10000 (here 2000).
speed_excess	'72/60'	Whole-number percentage the speed is over the limit: (72-60)*100//60 = 20. Limit must be > 0.

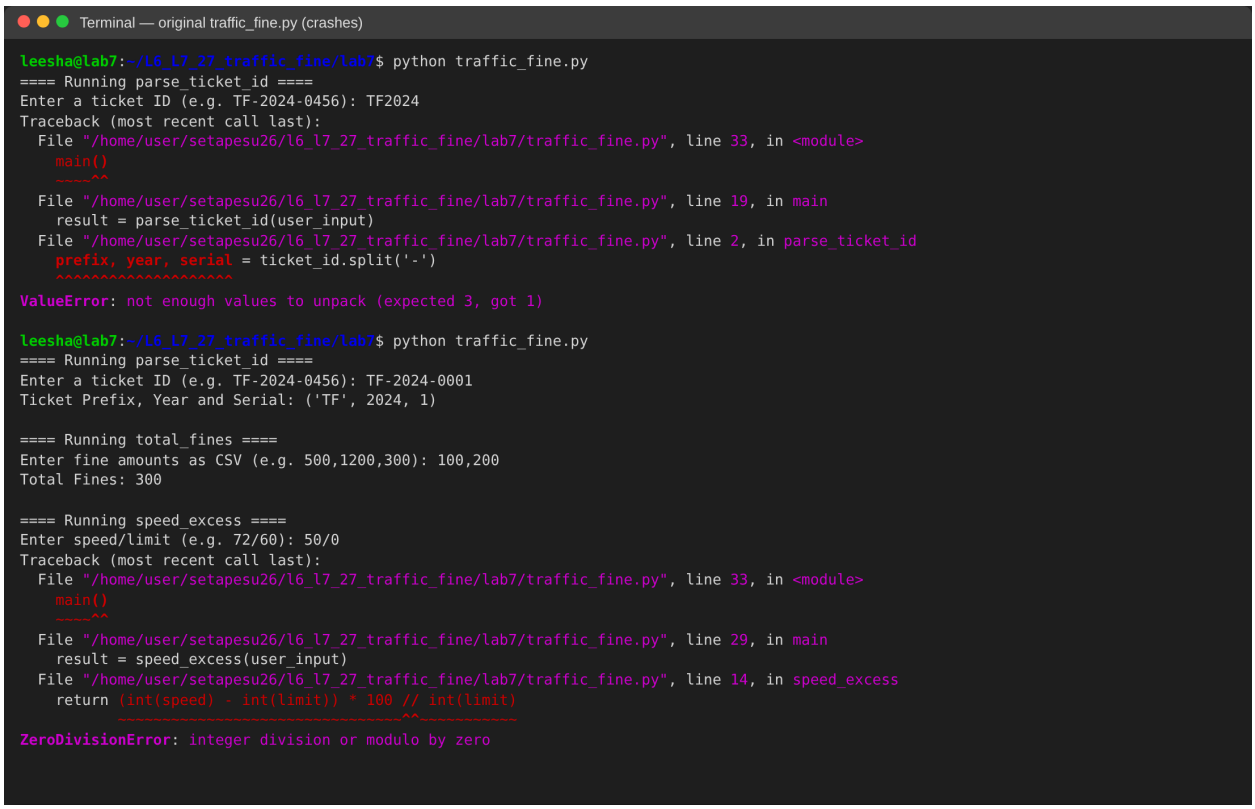
Chosen error-handling contract: the starter tests fuzz every function with `st.text() | st.none()` and are named ..._no_crash, so each function must return a value for every input. The fixed functions therefore return **None** for invalid input, and `main()` prints a friendly message instead of a traceback.

4. Original (buggy) application execution

The original program was run before any change (git status shows the file unmodified at commit 09d47d2). With the example inputs it works; with unusual input it gives wrong answers or crashes.



Screenshot 1a – Original code: correct for example input, but negative fines give a total of -5300 and an under-limit speed gives -17 % over the limit.



Screenshot 1b – Original code crashes: ValueError for ticket 'TF2024' and ZeroDivisionError for reading '50/0'.

5. Fuzz tests written (test_traffic_fine.py)

19 Hypothesis tests were written (6 for parse_ticket_id, 6 for total_fines, 7 for speed_excess). The three starter no_crash tests were kept and widened to fuzz with text, None, integers, floats (incl. NaN/inf), booleans, bytes, lists and dicts. Targeted property tests check normal inputs, boundaries, empty values, malformed formats, negative and zero values, Unicode digits and extremely long numbers. Two 'valid input' tests act as regression checks that correct inputs still give correct answers.

```
@given(st.integers(min_value=0, max_value=1000))
@example(0)
def test_speed_excess_zero_limit(speed):
    """A speed limit of 0 is invalid - must not raise ZeroDivisionError."""
    assert speed_excess(f"{speed}/0") is None
```

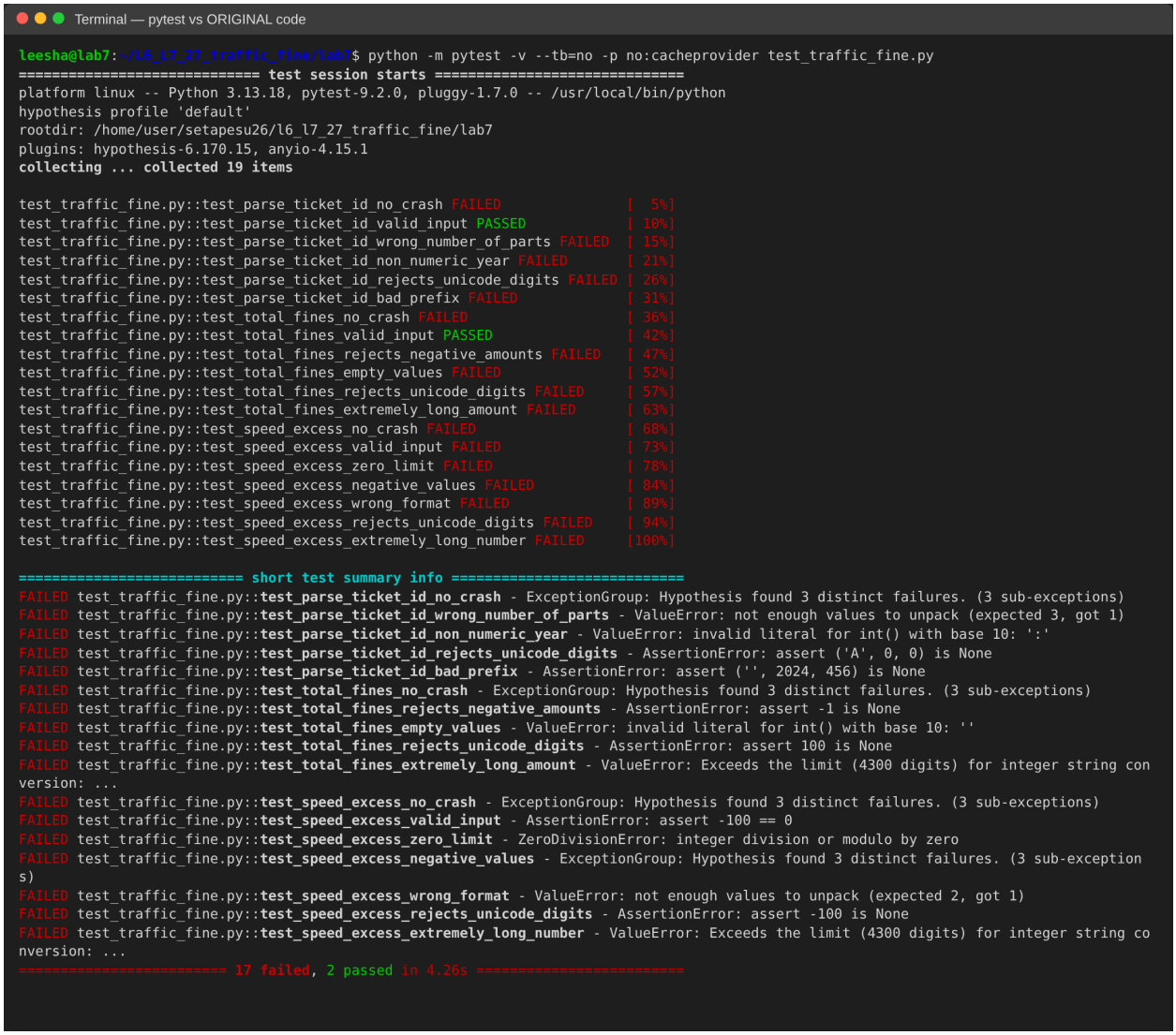
6. Bugs discovered by Hypothesis

#	Function	Falsifying input (found by Hypothesis)	Original behaviour	Problem
1	all three	None, b'' (bytes), ''	AttributeError / TypeError / ValueError	No type or empty-input check
2	parse_ticket_id	'TF2024', 'TF-2024-0456-99'	ValueError: not enough / too many values to unpack	Wrong number of '-' parts not handled
3	parse_ticket_id	'A-:-0'	ValueError: invalid literal for int()	Non-numeric year/serial crash
4	parse_ticket_id	'A-·····-0' (Arabic-Indic digits)	Returns ('A', 0, 0)	int() silently accepts Unicode digits
5	parse_ticket_id	'-2024-456'	Returns ('', 2024, 456)	Empty / non-letter prefix accepted
6	total_fines	'-1'	Returns -1	Negative fines accepted; total can be negative
7	total_fines	'', '500,', '500,,300'	ValueError: invalid literal for int()	Empty items crash
8	total_fines	'100,·'	Returns 100	Unicode digits accepted
9	total_fines	'9'*4301 (4301-digit amount)	ValueError: Exceeds the limit (4300 digits)	Extreme values crash
10	speed_excess	'0/0', '50/0'	ZeroDivisionError	Limit 0 not checked
11	speed_excess	'0/1', '-1/1', '0/-1'	Returns -100 / negative %	Negative values accepted; result 'over limit' can be negative
12	speed_excess	'72', '72/60/50'	ValueError: not enough / too many values to unpack	Missing or extra '/' not handled
13	speed_excess	'·/60'; '9'*4301 + '/60'	Returns -100; ValueError (4300 digits)	Unicode digits accepted; extreme values crash

Note on bug 11: for a speed below the limit the original returns a negative 'percent over limit' (e.g. 50/60 → -17). This report treats that as incorrect, since a driver under the limit is 0 % over it; the fixed function returns 0 in that case.

7. Failing Hypothesis tests against the original code

Running `python -m pytest -v test_traffic_fine.py` against the unmodified original code: **17 failed, 2 passed**. The 2 passing tests are the valid-input regression tests. The full log is saved in `evidence/pytest_original_full.txt`.



Screenshot 2a – pytest against the ORIGINAL code: 17 failed, 2 passed.



Screenshot 2b – Detail of four failures with the falsifying examples Hypothesis found (Unicode year, negative fine, 4301-digit amount, zero speed limit).

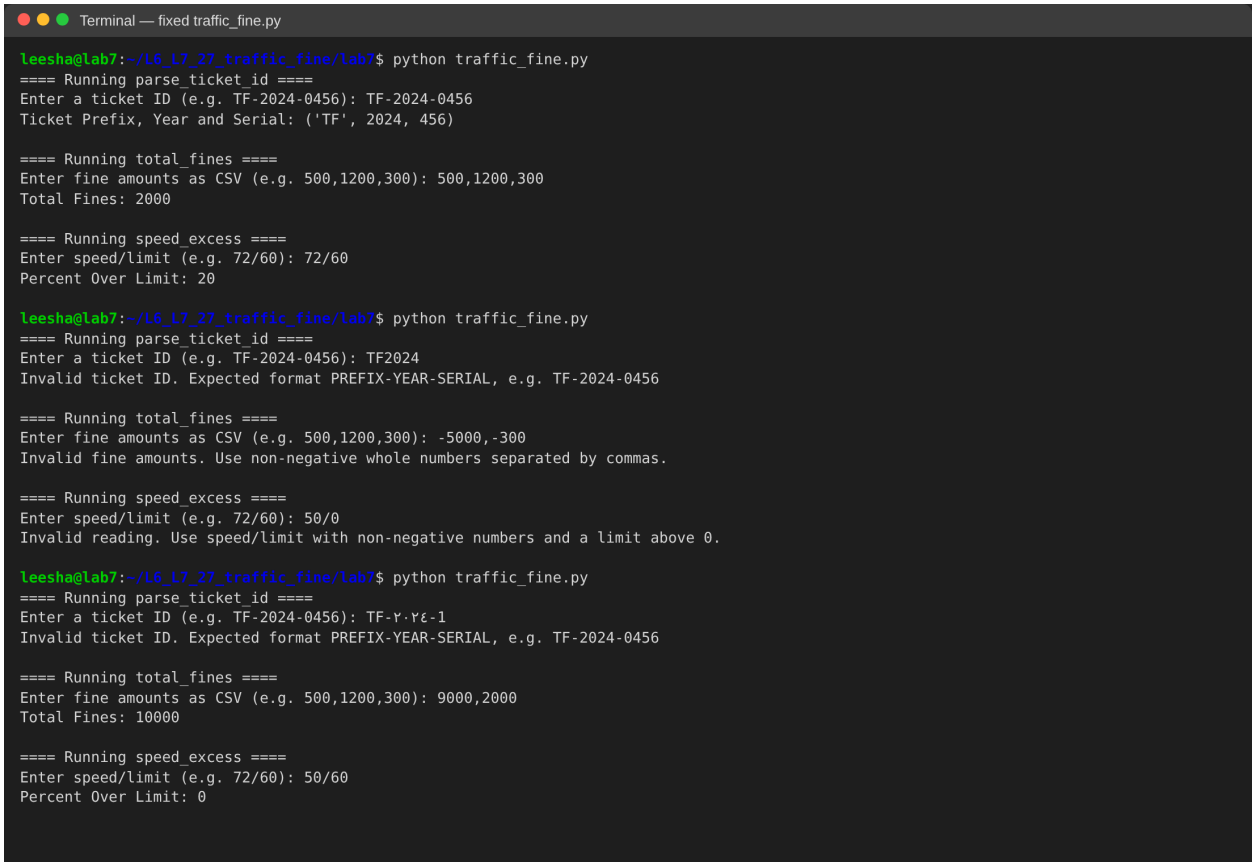
8. Fixes implemented

The original design (three functions + interactive `main()`) was kept. Changes:

Fix	Bugs fixed
Each function first checks `isinstance(x, str)`; otherwise returns None.	1
Split result is checked for exactly 3 parts (ticket) / 2 parts (speed) before unpacking.	2, 12
Helper `_is_ascii_number()` accepts only the characters 0-9 (`str.isdigit()` alone would still allow Unicode digits). This also rejects '-', '+', '.', spaces and empty strings.	3, 4, 6, 7, 8, 11, 13
Ticket prefix must be non-empty ASCII letters; year must be exactly 4 digits.	5
total_fines: amounts with more digits than the cap are treated as 'cap reached' without calling int(); speed_excess/serial use a safe `_to_int()` that returns None on the 4300-digit limit.	9, 13
speed_excess: limit 0 returns None; speed at/below the limit returns 0.	10, 11
main() prints a clear message when a function returns None instead of crashing.	all
Surrounding spaces are tolerated (' 72 / 60 ' → 20, '500, 1200' → 1700).	usability

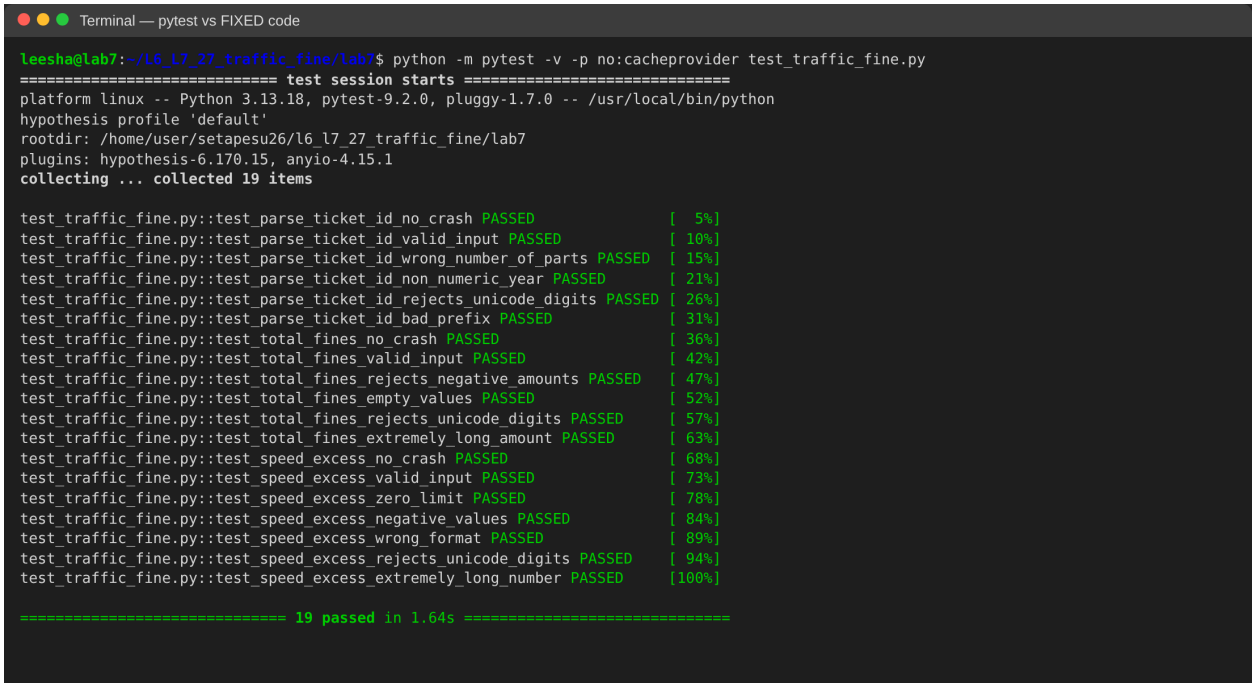
```
def speed_excess(reading):
    if not isinstance(reading, str):
        return None
    parts = reading.strip().split('/')
    if len(parts) != 2:
        return None
    speed_text, limit_text = parts[0].strip(), parts[1].strip()
    if not (_is_ascii_number(speed_text) and _is_ascii_number(limit_text)):
        return None
    speed = _to_int(speed_text)
    limit = _to_int(limit_text)
    if speed is None or limit is None or limit == 0:
        return None
    if speed <= limit:
        return 0
    return (speed - limit) * 100 // limit
```

9. Fixed application execution



Screenshot 3 – Fixed code: example inputs give the same results as before (TF/2024/456, 2000, 20); the inputs that crashed now print clear messages; the 10000 cap and 0 % under the limit work.

10. Final passing test run



Screenshot 4 – python -m pytest -v test_traffic_fine.py against the FIXED code: 19 passed, 0 failed.

As an extra check (not shown in a screenshot) the suite was also run with 5,000 Hypothesis examples per test (about 95,000 generated inputs): 19 passed. No test was weakened, skipped or removed to make the suite pass.

11. Conclusion and learning outcomes

The original traffic fine code worked for the 'happy path' examples in its prompts, but Hypothesis found 13 distinct problems within seconds: crashes on None, empty, malformed and extremely long input, a division by zero, and silently wrong results for negative values and Unicode digits. After adding input validation, all 19 fuzz tests pass and the program behaves correctly for both normal and unexpected input.

Learning outcomes

• Example-based testing with a few 'nice' inputs misses most real bugs; generated inputs explore edge cases a developer does not think of (e.g. Unicode digits, Python's 4300-digit int limit).
• Hypothesis shrinks failures to the smallest example (e.g. '-1', '0/1'), which makes the bug easy to understand and fix.
• Good fuzz tests assert the intended behaviour (properties such as 'total is between 0 and 10000'), not just 'does not crash'.
• Validate input at the boundary: check type, format, character set and range before converting.
• Keeping the valid-input tests as regression checks proved the fixes did not break correct behaviour.

Note on screenshots

The work was done in a cloud Linux container with no desktop. Each screenshot is real terminal output: the command was run in a pseudo-terminal (so typed input and pytest colours appear exactly as in a terminal), and the captured output was rendered to PNG in a headless browser. The window frame and the 'leesha@lab7' prompt text were added by the capture script; the command output itself is unedited. Plain-text copies of every run are in the `evidence/` folder.